

# Contents

|          |                                             |          |
|----------|---------------------------------------------|----------|
| <b>1</b> | <b>CAVS implementation</b>                  | <b>2</b> |
| 1.0.1    | Architecture details . . . . .              | 3        |
| 1.0.2    | Trust in AI Models . . . . .                | 4        |
| 1.0.3    | Testing AI tools . . . . .                  | 5        |
| 1.1      | Webapp and CAVS in detail . . . . .         | 6        |
| 1.1.1    | Certificate Issuer section . . . . .        | 6        |
| 1.1.2    | CAVS dashboard . . . . .                    | 7        |
| 1.1.3    | Author section . . . . .                    | 9        |
| 1.1.4    | Spreader section and VP Diffusion . . . . . | 14       |
| 1.1.5    | VCs and QRCode . . . . .                    | 16       |
| 1.1.6    | Reader section . . . . .                    | 16       |

# Chapter 1

## CAVS implementation

The code illustrated here is available on GitHub at the repository:caltr98/CAVS<sup>11</sup> (with execution instructions). Throughout the chapter. We implemented CAVS using a microservice architecture. The idea was to create a solution with many pieces that someone could chain and interchange with its alternative, given that it can interact with their REST interface.

The components run on Docker containers; the "Docker compose" command connects them automatically and creates a network for the CAVS whole system. The BERT model extracts keywords from each statement, using KeyBERT described in section. A microservice querying one model or the other is defined, where the keyBERT microservice runs on the local machine, while the microservice with GPT-3.5 queries the OpenAI APIs.

It may be possible to enrich the keywords by looking at related keywords or the general field of the keywords, and the author actor can do this when providing a statement to CAVS, which will call an internal microservice connected to the Yago knowledge base or another microservice using GPT-3.5, this latter task if performed in the keyLLM microservice.

For the keywords mapping to skills, we have a microservice running ojd\_daps\_skills model allows us to extract ESCO ontology-defined skills from the text.

---

<sup>11</sup><https://github.com/caltr98/CAVS>

CAVS uses a microservice with a Veramo Agent with its own DID. The Verano Agent microservice enables those tasks for different actors: **1.** Create DID with key-pairs, **2.** Import DID, **3.** Issue VC, **4.** Verify VC/VP, **5.** Store VC, **6.** Create VP.

In particular, the CAVS system allows the verification of the ESCO Skills VC and then the creation of a VC Statement. This allows to fulfill the requisite of portable VCs for and indication of credibility.

For the author, it allows the storing of the VCs mentioned above and the creation of VPs Diffusion, the latter also for Spreaders.

For readers, it allows to verify the VPs and the VC Statement to which the VPs Diffusion refers.

### 1.0.1 Architecture details

We may see the system's architecture in Figure 1.1. All the microservices

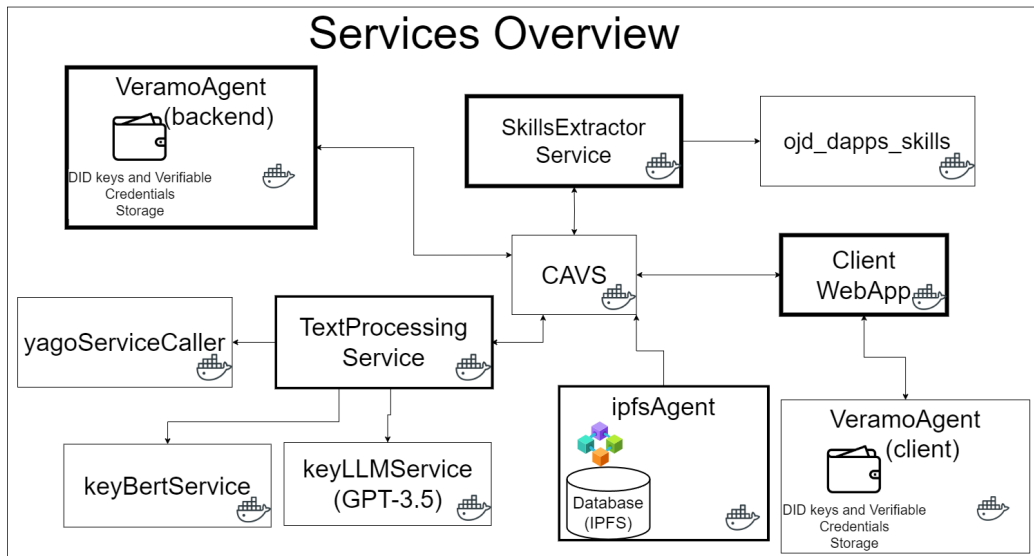


Figure 1.1: An overview of the microservices we created for the CAVS system and their interconnection. In thick lines, the microservices are directly connected to the CAVS microservice

besides CAVS are stateless, allowing them to fail, and a daemon process or a person can restart them without issues. The Client Webapp can change the state of the CAVS microservice, which includes:

- the selected engine for keyword extraction between keyBERT and its subconfigurations. In the microservice keyBertService, only the "all-MiniLM-L6-v2" BERT model is used for skill extraction, which is set up to extract at most five keywords of one token length and five keywords of two token lengths. Meanwhile, in keyLLMService, two configurations can be called in requests, including a mixture of BERT and GPT-3.5 and another only calling GPT-3.5 in keyLLMService. Both microservices use KeyBert library, **they functionalities are not in the same microservice**, but separated in two microservices, because we expect KeyBertService to be the most widely used as it just runs on the local machine, without costs incurred in calling OpenAI APIs. Given this, we created the KeyBert microservice that only runs the local BERT model, keeping the microservice lightweight. Changing the engine will change the REST API calls that CAVS performs to TextProcessingService and, in turn, the REST API calls it performs and on which endpoint.
- the selected engine for skill extraction, which is currently only ojd\_daps\_skills. Changing the engine will change the REST API calls that CAVS performs to SkillExtractorService

Since ojd\_daps\_skills and keyBert are models with libraries written in Python, the microservices enclosing them were written in Python. The microservice interfacing with OpenAI GPT-3.5 is also written in Python, standardizing the interaction with AI tools via Python. We use Python's Flask framework to serve HTTP requests for said microservices.

### 1.0.2 Trust in AI Models

To allow the verification of results from TextProcessingService and SkillExtractorService, we include a checksum of the trained base models used by the tools inside the VC Statement. This is fundamental to satisfy requisite **(R2)**, as the reader may run the tools themselves to make their judgments. In particular:

- keyBert: we include the all-MiniLM-L6-v2 hash of the model state dictionary with trained parameters (weights and biases) and architecture of the NN model. The checksum is generated with sha256.
- keyLLM: we provide the checksum above for extraction with the

BERT model in conjunction with the LLM model version, which in our case will be a version of OpenAI GPTs available API callable models. Furthermore, if keyLLM is used to enrich keywords, a link to the prompt, with the GPT version used and the parameters for each call, will be provided.

- `ojd_daps_skill`: for this, we make two checksums for two models used by the tool, one as before for the BERT model used for the mapping skills extracted in the toy taxonomy to a reference taxonomy and the other for the extraction of the skill to the toy taxonomy for the NER model. For the NER model, we hash the `config.cfg` file.

### 1.0.3 Testing AI tools

Note that the web app called TextWebApp at `localhost:13001` (when executing the command `Docker compose`) allows testing all the AI text processing tools independently from the SSI concepts in CAVS. Furthermore, we may see the interface of the web app in Figure 1.2 and Figure 1.3

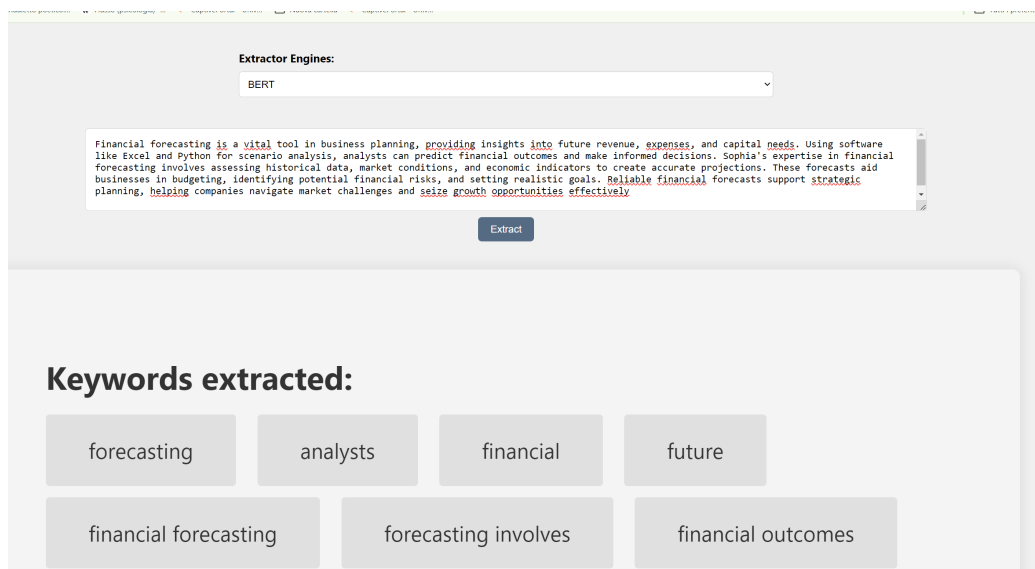


Figure 1.2: The TextWebApp section that allows to extract keywords from text

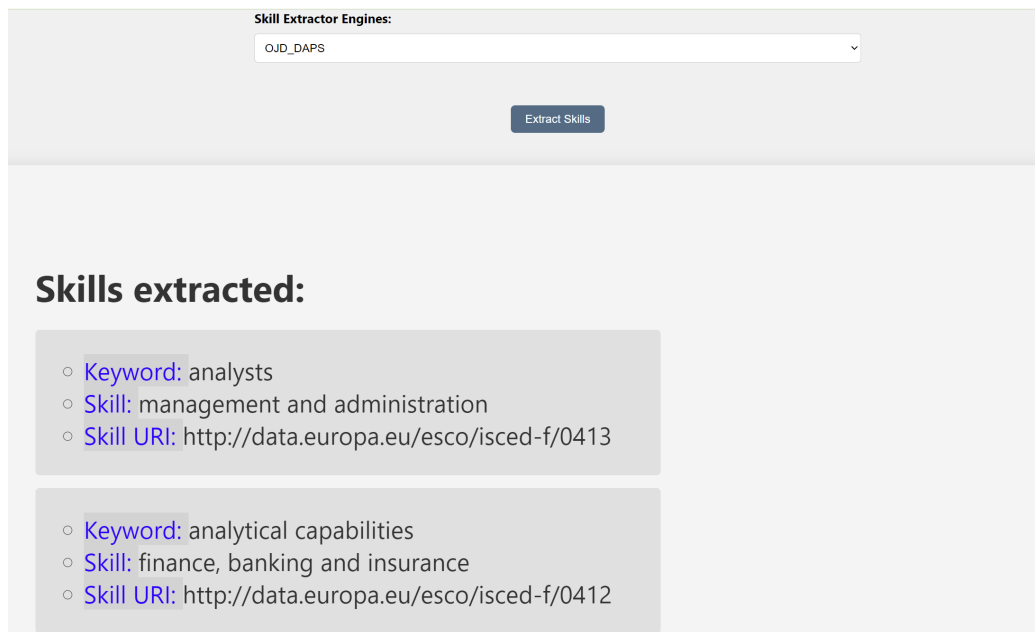


Figure 1.3: The TextWebApp section that allows to map keywords to skills

## 1.1 Webapp and CAVS in detail

We designed a web app that serves as a client for the microservices illustrated.

It allows the generalization of a DID for the Sepolia testnet, a testnet of Ethereum, but also to import a DID of the DID method ETH mentioned in section ?? by importing it in the Wallet address.

### 1.1.1 Certificate Issuer section

Figure 1.4 shows the dashboard for the CertificateIssuer, which grants ESCO Skills VCs, with the skills extracted from a text or added manually. The field is composed of:

- New Holder DID: the DID of the Holder to be assigned the certificates
- New Holder Endpoint DID: the endpoint URL to send the ESCO

Skill VC after it is created; it is the VeramoAgent microservice on the client side.

- **Certificate Description:** a description of the certificate to assign, allowing the extraction of ESCO Skills from said description using the CAVS microservice.
- Not seen in Figure, there are fields to input the ESCO Skill manually, with their name and URI.

### 1.1.2 CAVS dashboard

The screenshot displays the 'Issue Certificate Verifiable Credential' section of the CAVS dashboard. It includes the following elements:

- New Holder DID:** A text input field.
- New Holder Endpoint URL:** A text input field.
- Certificate Description To Skills:** A large text area with the placeholder text 'Enter Certificate Description...'. A small icon in the bottom right corner indicates that rich text formatting is available.
- Keyword Extractor Engines:** A dropdown menu currently set to 'BERT'.
- Skill Extractor Engines:** A dropdown menu currently set to 'OJD\_DAPS', followed by a blue button labeled 'Extract Skills from Description'.

Figure 1.4: The section of the Web app for CertificateIssuer

The CAVS dashboard is intended to be its web app, which the system administrator uses. We included it in the whole web app to showcase the system features. In this section, we may set up the CAVS microservice backend with the engines for VC Statement, keyword extraction, synonyms generator with general field extraction, and skills extraction. As shown in the figure, we can choose the configuration and see the current one 1.5. The possible choices are:

**CAVS available at <http://localhost:4200>**

**Extractor Engines:**

BERT

**Enricher Engines:**

YAGO

**Skill Extractor Engines:**

OJD\_DAPS

**Update Config**

Current Setup: BERT, NONE, OJD\_DAPS

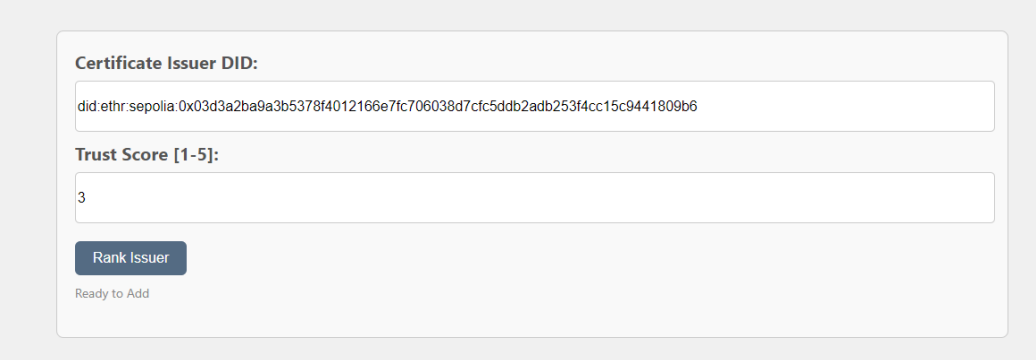
Figure 1.5: The section of the Web app for CAVS setup

- Extractor Engines: BERT, BERT+LLM, LLM. Where LLM refers to GPT-3.5, BERT refers to the microservice keyBertService; BERT+LLM refers to the microservice on keyLLMService.
- Enricher Engines: Yago and GPT, two microservices calling the online APIs of each tool.
- Skill Extractor Engines: OJD\_DAPS\_SKILL (Nesta's skill extractor) microservice serving requests to said tool.

### Trust Certificate Issuer

Given that not all Certificate Issuers are to be trusted as authoritative in issuing a certificate, we provide a trust score to assign as a solution to them. The system administrator can review a Certificate Issuer and may input their DID and a score from 1 to 5 assigned to it, which the CAVS system will consider during VC Statement creation.

We may see in the figure the interface that allows this task 1.6



The figure shows a web application interface for adding a Certificate Issuer. It contains two input fields: 'Certificate Issuer DID:' with a text input containing a long hexadecimal string, and 'Trust Score [1-5]:' with a text input containing the number '3'. Below these fields is a blue button labeled 'Rank Issuer'. At the bottom left of the form area, the text 'Ready to Add' is displayed.

Figure 1.6: The section of the Web app for adding the DID of Certificate Issuer and their ranking

### 1.1.3 Author section

The author of a statement using the functionalities of CAVS may obtain a VC Statement. To request one, the author presents the ESCO VC Skill to CAVS. This can be seen in Figure 1.7, then the author provides the statement, the URL of where the statement is hosted, and the statement category, which can be seen in Figure 1.8. We may know the structure of the VC Statement created after the request of the previous figure in

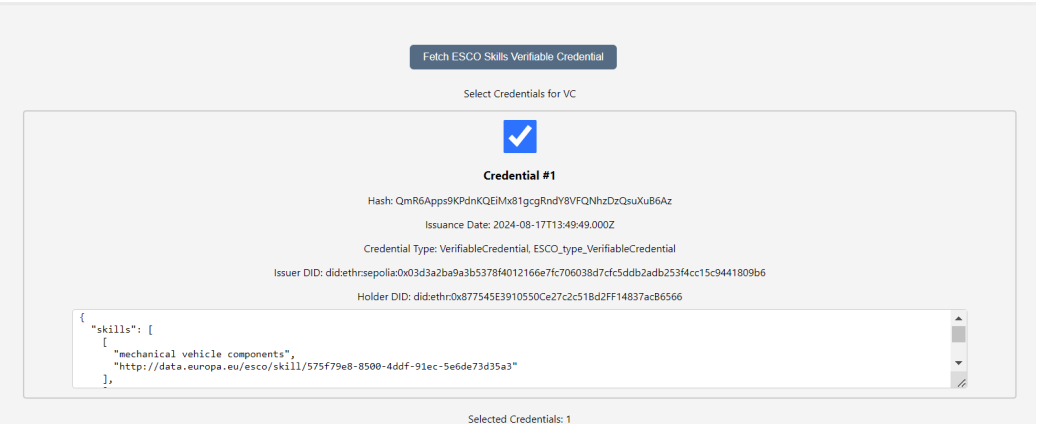


Figure 1.7: The section of the Web app that the author uses to provide the statement to present to CAVS

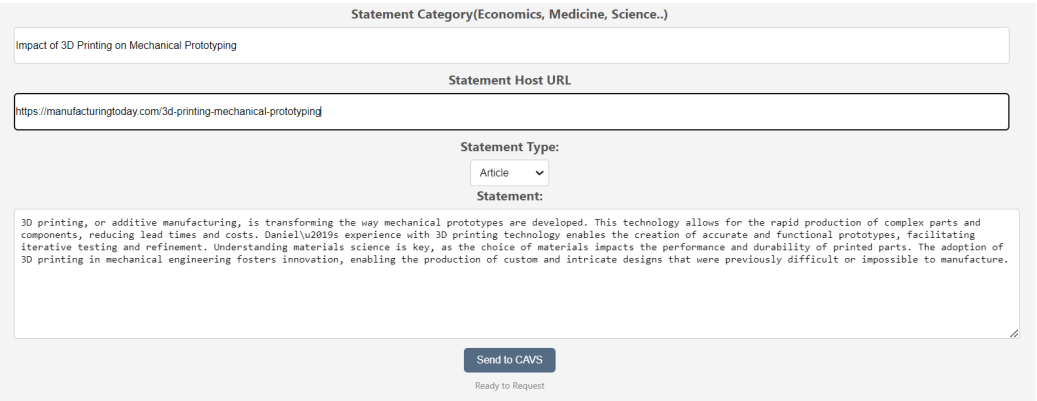


Figure 1.8: The section of the Web app that the author uses to see their ESCO VC Skill and choose to present to CAVS

the JSON 1.1. It can be noticed how there are the keywords extracted, and the ESCO VC Skill with the skills that matched said keywords.

The ESCO VC Skills are placed in the following categories:

- **Direct Keyword Match:** inside **credentials\_for\_skills**
- **Synonyms Match:** inside **credentials\_for\_similar\_concepts\_skills**
- **General Field Match:** inside **credentials\_for\_general\_concepts\_skills**

The latter two categories are optional and only used if CAVS is set up to obtain the synonyms and extract the general field. Each category has an array of ordered rankings of the issuers based on the issuers' provided DID rankings.

Listing 1.1: VC Statement containing information about the configuration and keywords used for skills verification

---

```

1 {
2   "cavs_config": "BERT+NONE+OJD_DAPS",
3   "keywords": ["manufacturing", "3d", "mechanical", "printing", "materials", "prototypes",
4     "technology", "engineering", "designs", "3d printing", "additive manufacturing", "
5     manufacturing is ", "understanding materials", "of 3d", "prototypes are ", "
6     intricate designs", "engineering", "way mechanical"],
7   "keywords_model": {
8     "model_checksum": "16c9297ae82f8565b5369459db1b7f310e719798d1d13145d349125ca58cf84d",
9     "name": "all-MiniLM-L6-v2"
10  },
11  "skills_model": {
12    "OJD_DAPS_BERMODELCHECKSUM": "16
13      c9297ae82f8565b5369459db1b7f310e719798d1d13145d349125ca58cf84d",
14    "OJD_DAPS_BERTMODELNAME": "sentence-transformers/all-MiniLM-L6-v2",
15    "OJD_DAPS_NERMODELCHECKSUM": "398
16      f0be769f8b1e017bed2e93372ca2a7b74dddbe66f27063d0cf5de06c7a9fe",
17    "OJD_DAPS_NERMODELNAME": "spacy_ner_model"
18  },
19  "similar_concepts_keywords": [],
20  "general_concepts_keywords": [],
21  "statementType": "News",
22  "statementCategory": "Mechanics",
23  "statementHostURL": "mechanicsonlinewebsite.com",
24  "credentials_for_skills": [
25    {
26      "issuanceDate": "2024-08-23T16:42:20.000Z",
27      "@context": ["https://www.w3.org/2018/credentials/v1", "https://www.w3.org/ns/
28        credentials/v2"],
29      "type": ["VerifiableCredential", "ESCO_type_VerifiableCredential"],
30      "credentialSubject": {
31        "skills": [
32          ["engineering", "http://data.europa.eu/esco/skill/b8ac179c-4a29
33            -4467-9177-031daf1321d6"],
34          ["CAD software", "http://data.europa.eu/esco/skill
35            /a2b5dcf3-5b6a-453d-876c-cff540c0faf1"],
36          ["a manufacturing process", "http://
37            data.europa.eu/esco/skill/3786b61f-f22e-48d1-af8d-ad4c354534db"],
38          ["materials

```

After obtaining the VC Statement, an author may publish it to IPFS; as shown in Figure 1.9, the CID is obtained after publishing it. This is fundamental to satisfy the requisite **(S1)** for the Spreader to have a way of proving the trustworthiness of their sources.

Using the CID of the VC Statement, the author can create a VP Diffusion, which the system requires to create and distribute alongside the statement. In this case, the VP Diffusion has the type "origin," as it's the start of the chain of VP Diffusion directing to the VC Statement. We may see a VC Diffusion of type origin created in Figure 1.10.

[Publish/Get on IPFS](#)

**Credential #1**

**CID is defined: bafkreifbxf2wh6tdazinfijktsvfa4g427jpxkl7iq4y3ggczks2aydiy**

Hash: QmeaWMZT5RTcPFksaJWZ333DvYgLuNBKYWY5im6wUKfjEM

Issuance Date: 2024-08-19T14:42:00.000Z

Credential Type: VerifiableCredential, StatementVerifiableCredential

Issuer DID: did:ethr:0x877545E3910550Ce27c2c51Bd2FF14837acB6566

Holder DID: did:ethr:sepolia:0x03d3a2ba9a3b5378f4012166e7fc706038d7cfc5ddb2adb253f4cc15c9441809b6

```
{
  "keywords": [
    "engineering",
    "cad",
    "training",
    "proficiency",
  ]
}
```

Figure 1.9: The section of the Web app that the author uses to publish a VC Statement on IPFS or obtain its CID if it has already been published

[illegible]

Figure 1.10: The section of the Web app that the author uses to create a VP Diffusion of type origin

#### 1.1.4 Spreader section and VP Diffusion

[illegible]

Figure 1.11: The section of the Web app that Spreader uses to create a VP Diffuusion of type diffusion

A Spreader reshapes an original statement or a previous resharing of it. To do so, it uses the JWT of the VP Diffusion, which it found last in the chain of resharing, and uses it to create their VP Diffusion of type diffusion rather than origin. This corresponds to the requirement **(S2)** of creating a chain from the Spreader source to the original one. The JWT represents the result of the signature from the creator of the VP Diffusion, as the proof of the signature is an embedding proof that creates a hash of the VP, which can be reversed to the plain-text VP.

We may see in Figure 1.11 how it does so, providing the JWT, the CID of the VP Diffusion, and the UUID of the previous VP Diffusion. The UUID is integrated to provide unique identifiers to VP Diffusions in case they are published elsewhere rather than IPFS and so without a unique CID. For example, VP Diffusions may be hosted on web pages as QR codes or plain JSON rather than published on IPFS. We may see an

example of VP Diffusion of type diffusion in the listing 1.1.4

---

```

1  {
2    "verifiablePresentation" :{
3      "verifiableCredential" :[
4        {
5          "credentialSubject" :{
6            "statement_vc_cid_or_url":"
              bafkreihi4lfunkhwcw4x5xpxk7u4k3sifc6nnvqgtxyrsvfzfyavnlamb4",
7            "prev_vp_uuid":"00c86dfe-b1d2-496d-b446-030f751e078e",
8            "url_or_cid_jwt_prev":"
              bafkreifdavmckmgzdzbdzembynm6rb4dpvos2yf267m5mzqjtzpwnr7ru",
9            "type": "diffusion",
10           "host_URL":"facebook.com/post/...",
11           "assertion": "This VP is submitted by the subject as evidence of VC
              propagation",
12           "id": "46ea4b57-698e-4d40-8fc7-f5c834d2124d"
13         },
14         "issuer" :{
15           "id": "did:ethr:0x877545E3910550Ce27c2c51Bd2FF14837acB6566"
16         },
17         "type" :[
18           "VerifiableCredential",
19           "VCPresentation",
20           "VCforStatementDiffusion_VPv01"
21         ],
22         "@context":[
23           "https://www.w3.org/2018/credentials/v1",
24           "https://www.w3.org/ns/credentials/v2"
25         ],
26         "issuanceDate": "2024-08-17T15:09:15.000Z",
27         "proof" :{
28           "type": "JwtProof2020",
29           "jwt": "eyJ...."
30         }
31       ]
32     },
33     "holder": "did:ethr:0x877545E3910550Ce27c2c51Bd2FF14837acB6566",
34     "id": "46ea4b57-698e-4d40-8fc7-f5c834d2124d",
35     "type" :[
36       "VerifiablePresentation",
37       "StatementDiffusion_VPv01"
38     ],
39     "@context":[
40       "https://www.w3.org/2018/credentials/v1",
41       "https://www.w3.org/ns/credentials/v2"
42     ],
43     "issuanceDate": "2024-08-17T15:09:16.000Z",
44     "proof" :{
45       "type": "JwtProof2020",
46       "jwt": "eyJhb...."
47     }
48   }
49 }

```

---

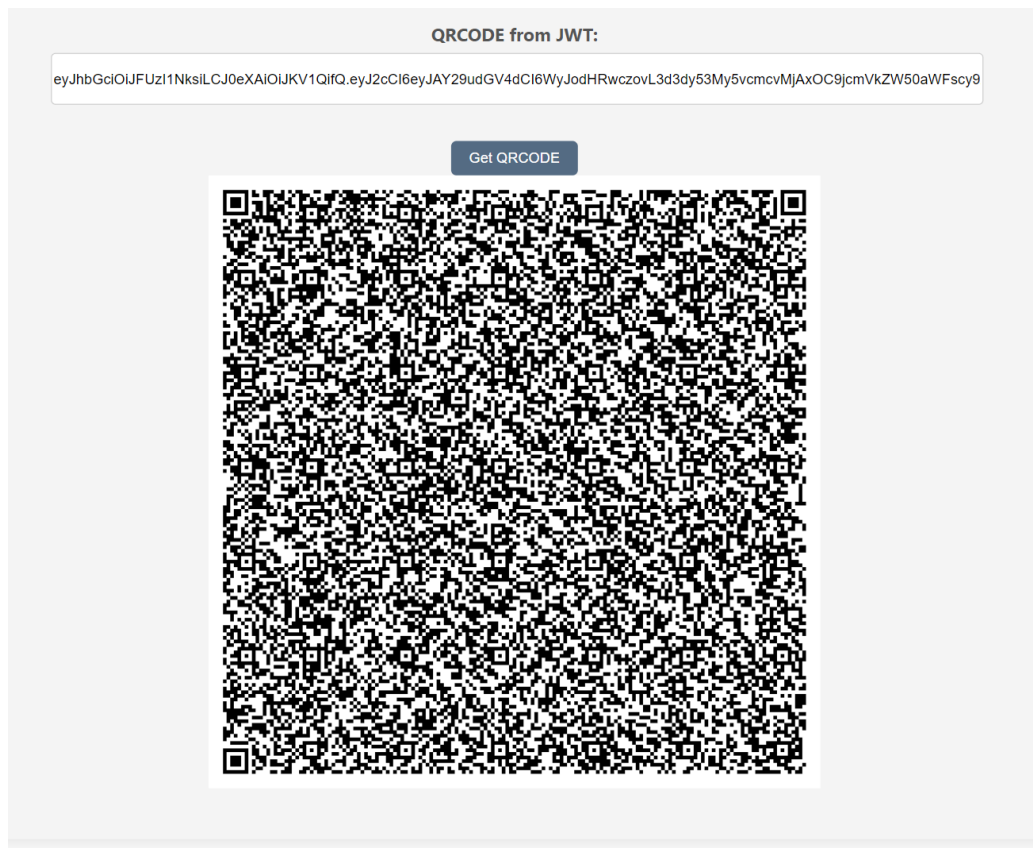


Figure 1.12: A QR CODE of a VP Diffusion

### 1.1.5 VCs and QRCode

### 1.1.6 Reader section

The dashboard for the reader, which can be seen in Figure 1.13, allows it to:

- decode a VP Diffusion from a JWT, possible since its signature is embedding proof that can be reversed to the plain-text VP. Details on embedding proofs are in subsection ??.
- decode a VP Diffusion from the image of a QR CODE generated from a JWT, said JWT(

Given that the VCs and VPs are JSONS with a JWT, that is the signature made by the issuer in case of a VC or the VP creator. That signature is a hash of the VCs/VPs, which can be reverted to the plain-text version.

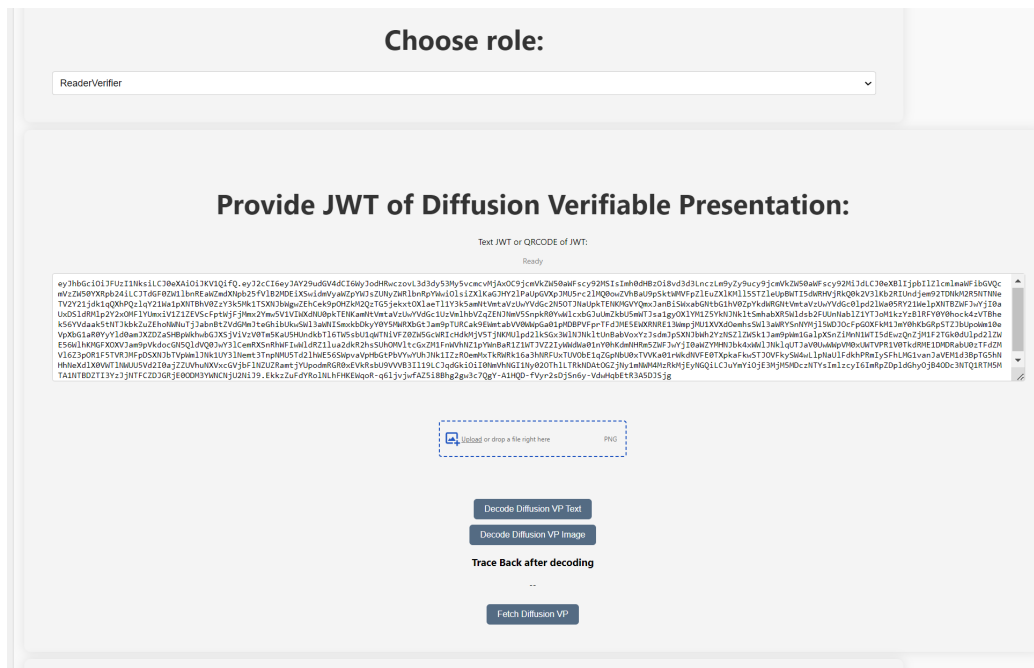


Figure 1.13: Reader section showing how they may decode a VP Diffusion from a QR CODE or a JWT

From the JWT, creating a QR code for them is possible. For example, we may create the QRcode of a VP Diffusion and place it on a webpage alongside a statement. A spreader or an author may place a QRCode on a webpage. We see the section dedicated to the QR CODE creation a QR CODE generated for a VP Diffusion in Figure 1.12. Then, after having the VP Diffusion decoded, the reader may:

- Trace back the VP Diffusion, following the URL or CID chain of VP Diffusions up to the VP Diffusion of type origin
- Fetch the VC Statement generated by CAVS, which includes the keywords extracted, the skills, and the ranking of the ESCO Skill VC given by the Certificate Issuer to the statement author. This can be seen in Figure 1.15.
- Verify all the VP Diffusions, VC Statement recovered, and the ESCO Skill VC inside the VC Statement, as can be seen in Figure 1.15
- The verification after having traced back VPDiffusions can be seen in Figure 1.14

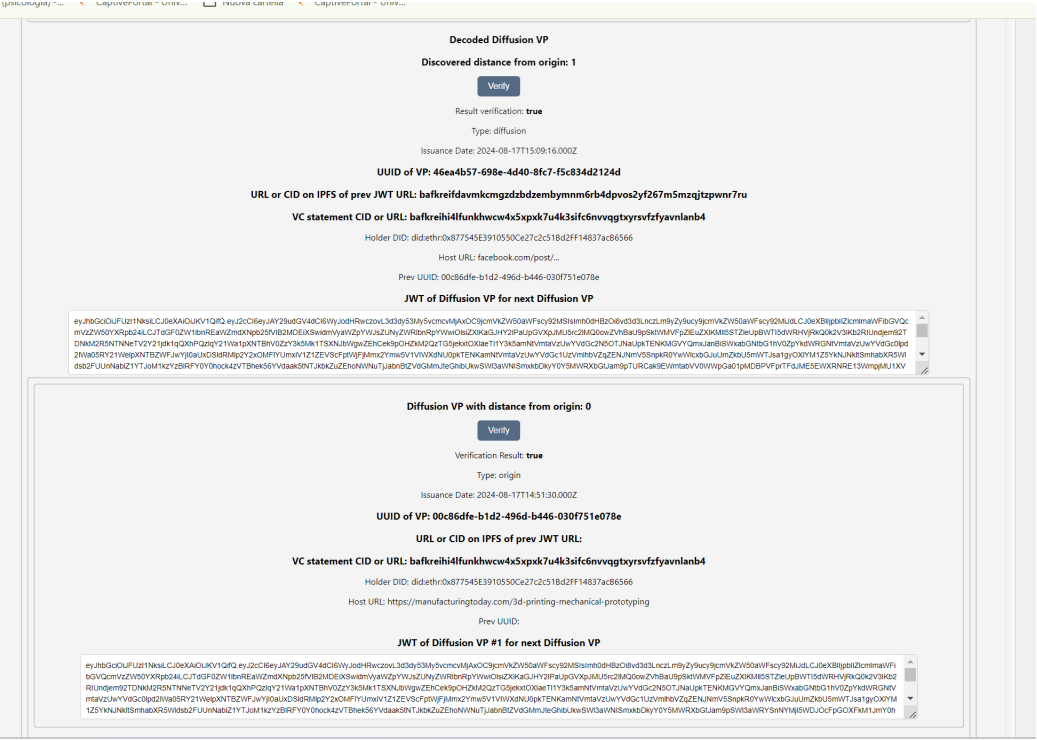


Figure 1.14: the traceback outcome of VP Diffusion and the result of their verification

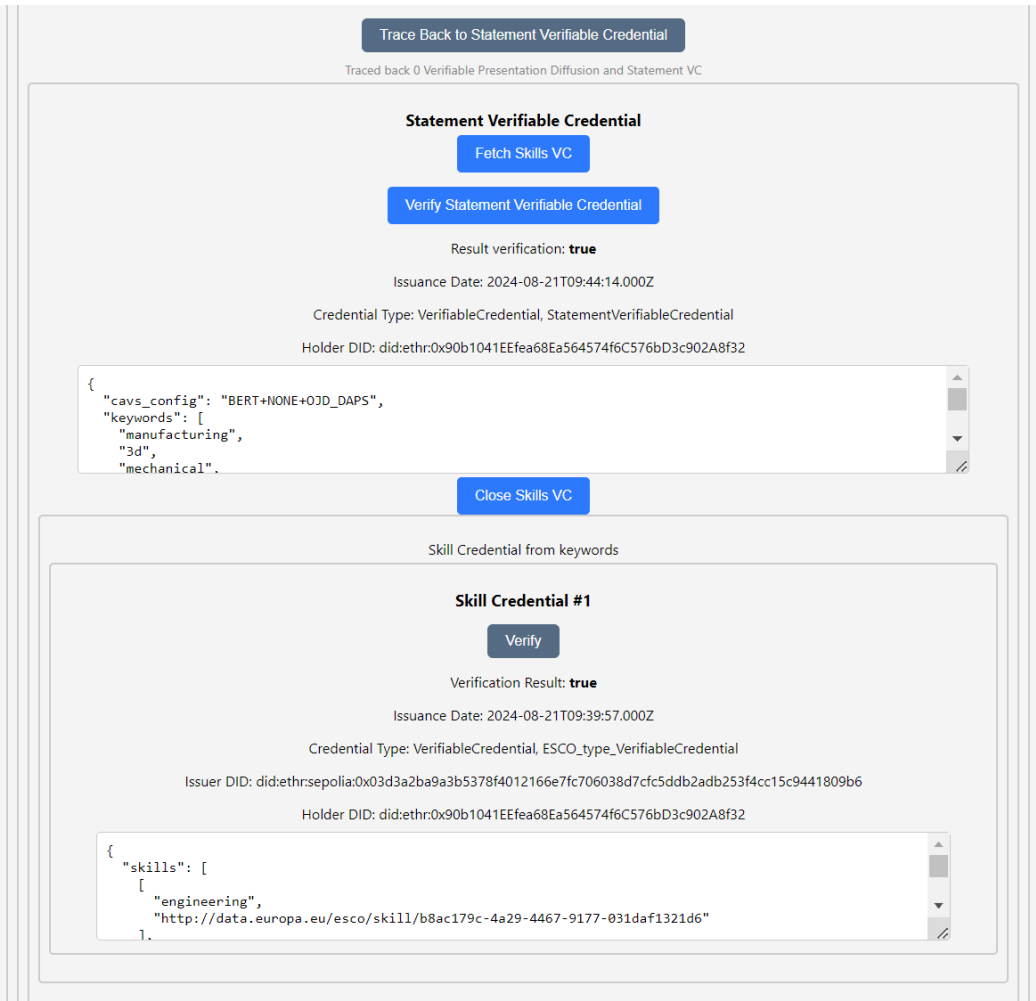


Figure 1.15: A fetched VC Statement with one ESCO Skill VC that was provided to CAVS for its creation